



Growing Code

ガーデンデータを通じたPythonの基礎

概要： コミュニティガーデンのデータを分析しながら、Pythonの基本的な構文と意味論を学びます。持続可能な地域活動に貢献しつつ、式、変数、関数、制御構造について学びます。

バージョン：3.0

目次

I	はじめに	2
II	AIの指示	3
III	はじめに	5
IV	一般的な注意事項	6
V	演習 0：Hello Garden	7
VI	演習1：ガーデンの名前	8
VII	演習2：庭の区画面積	9
VIII	演習3：収穫総量	10
IX	演習4：植物の年齢確認	11
X	演習5：水やりのリマインダー	12
XI	演習6：収穫までの日数カウント	13
XII	演習7：種子の在庫管理（種類による注釈付き）	15
XIII	参考資料	17
XIV	提出と提出方法	18

第I章 はじめに

世界中のコミュニティガーデンでは、データが成長、協力、そして影響力の物語を語っています。

地元の家族を養う収穫量の追跡から、貴重な資源を守るための水使用量の監視に至るまで、あらゆる測定値が重要です。それぞれのデータポイントは、誰かの献身を表しています——ボランティアが週末に費やした時間、子供が初めて育てたトマト、何十年にもわたる園芸の知恵を分かち合う年配の隣人などです。

Pythonは、こうした菜園と同じように、単純な種から力強く、栄養豊かなものへと成長しています。「モンティ・パイソンのフライング・サーカス」にちなんで名付けられたこの言語は、学びとは喜びに満ち、誰にでも親しみやすく、包摂的であるべきだということを私たちに思い出させてくれます。今日、Pythonは科学者が気候変動を理解するのを助け、地域社会が資源の共有を最適化できるようにし、そして誰もが生のデータを意味のある洞察へと変える力を与えています。

このプロジェクトでは、実際のコミュニティガーデンのデータを分析しながら、Pythonの基礎となる要素を学びます。プログラミングはガーデニングと同じように、コードの成長と、私たちが奉仕するコミュニティの成長の両方を育むことであることを理解できるでしょう。

第II章

AIの操作手順

● 背景

学習の過程において、AIはさまざまなタスクを支援してくれます。時間をかけて、AIツールの多様な機能や、それらがどのようにあなたの作業をサポートできるかを探ってみてください。ただし、常に慎重に扱い、結果を批判的に評価するようにしてください。コード、ドキュメント、アイデア、技術的な説明のいずれであっても、あなたの質問が適切に構成されていたか、生成された内容が正確であることを完全に確信することはできません。仲間たちは、間違いや見落としを防ぐための貴重なリソースです。

● 主なメッセージ

- ✦ AIを活用して、反復的または面倒な作業を軽減しましょう。
- ✦ 将来のキャリアに役立つ、コーディングおよび非コーディングの両面におけるプロンプティングスキルを磨きましょう。
- ✦ AIシステムの仕組みを理解し、よくあるリスク、バイアス、倫理的な問題をよりの確に予測・回避するようにしましょう。
- ✦ 仲間と協力し、技術スキルとリーダーシップスキルの両方を磨き続けましょう。
- ✦ 自分が完全に理解し、責任を負えるAI生成コンテンツのみを使用しましょう。

● 学習者のルール：

- AIツールをじっくりと探求し、その仕組みを理解することで、倫理的に活用し、潜在的なバイアスを軽減できるようにしてください。
- プロンプトを入力する前に、課題について熟考してください。これにより、正確な語彙を用いて、より明確で詳細かつ適切なプロンプトを作成できるようになります。
- AIによって生成されたものはすべて、体系的に確認、見直し、疑問を持ち、検証する習慣を身につけてください。

- 常にピアレビューを求めるようにしてください。自分自身の検証だけに頼ってはいけません。

● フェーズの成果：

- 汎用的なプロンプティングスキルと、特定の分野に特化したプロンプティングスキルの両方を身につける。
- AIツールを効果的に活用して生産性を向上させる。
- 計算的思考、問題解決能力、適応力、および協働能力を引き続き強化する。

● コメントと例：

- 試験や評価など、真の理解を示さなければならない状況に頻繁に直面することになります。準備を整え、技術的スキルと対人スキルの両方を磨き続けましょう。
- 自分の考え方を説明したり、仲間と議論したりすることで、理解の不足が明らかになることがよくあります。仲間との学びを優先しましょう。
- AIツールは、多くの場合、あなたの具体的な文脈を把握しておらず、一般的な回答を提供しがちです。同じ環境を共有する仲間なら、より適切で正確な洞察を提供してくれるでしょう。
- AIが最も可能性の高い答えを生成する傾向にあるのに対し、仲間は別の視点や貴重なニュアンスを提供してくれます。質を確かめるチェックポイントとして、仲間を頼りにしましょう。

✓ 良い実践例：

AIに「ソート関数をテストするにはどうすればいいですか？」と尋ねます。AIはいくつかのアイデアを提示してくれます。それらを試してみて、結果を仲間と確認します。そして、二人でアプローチを洗練させていきます。

✗ 悪い実践例：

AIに関数全体を作成させ、それを自分のプロジェクトにコピー＆ペーストする。ピア評価の際、その関数が何をするものか、なぜそうするのかを説明できない。信頼を失い、プロジェクトに失敗してしまう。

✓ 良い実践例：

AIを活用してパーサーの設計を支援します。その後、仲間と一緒にロジックを検証します。2つのバグを見つけ、協力して書き直します。その結果、より良く、よりクリーンで、完全に理解されたコードが完成します。

✗ 悪い実践例：

プロジェクトの重要な部分のコードをCopilotに生成させた。コンパイルは通ったが、パイプの処理方法を説明できなかった。評価の際、その正当性を説明できず、プロジェクトに失敗した。

第III章 はじめ

に

『Growing Code』へようこそ！

このプロジェクトでは、コミュニティガーデンを舞台にしたシナリオを通じて、Pythonの核心となる概念を学びます。実践的な演習に取り組みながら、魅力的で現実的な文脈の中でプログラミングの基礎概念を習得していきます。

各演習は前の演習を土台としており、有意義な課題に取り組む中で、プログラミングに不可欠なスキルを身につけることができます。



重要：各演習では、関数のみ（メインプログラムではない）を記述してください。各ファイルには、入力と出力を直接処理する、指定された関数のみを含めてください。if name == "_main_ ": といったブロックを記述しないでください。また、ファイル内で関数を直接呼び出したり、関数外にコードを記述したりしないでください。



補助ツール：演習のテストを支援するため、添付ファイルに main.py ファイルが用意されています。このファイルを作業ディレクトリにコピーし、python3 main.py を実行すると、関数を簡単にテストできます。この補助ツールは、関数を自動的にインポートしてテストを行います。

第IV章

一般的な注意事項

- 関数は Python 3.10 以降で記述してください。
- コードは flake8 リンターの基準に準拠している必要があります。
- 各演習問題は、それぞれ個別のファイルに記述してください。
- 各ファイルには、指定された関数のみを含めてください。
- 関数名は、指定されたものと完全に一致させる必要があります。
- 明示的に指定されていない限り、入力の妥当性チェックやエラー処理を行う必要はありません。
- 負の数や無効な入力に対しては、挙動は未定義となります（これらのケースを処理する必要はありません）。
- 型ヒントは任意ですが、演習 0 から 6 では学習の目的で推奨されます。演習 7 では必須です。



Growing Code に関する特記事項：これは基本的な Python 構文に焦点を当てた入門プロジェクトであるため、個々の Python 関数ファイル（例：`ft_hello_garden.py`、`ft_plot_area.py` など）のみを提出してください。高度なプロジェクト管理ツールについては、今後のプロジェクトで紹介します。

第V章

演習 0: Hello Garden

	演習0
	ft_hello_garden
	ディレクトリ: ex0/
	提出ファイル: ft_hello_garden.py
	許可されている: print()

初めてのPython関数へようこそ！「ft_hello_garden」という名前の関数を記述し、ガーデンコミュニティへの歓迎メッセージを表示してください。

```
>>> ft_hello_garden() こんにちは  
、Gardenコミュニティの皆さん！
```



メインプログラムではなく、関数 `ft_hello_garden()` のみを作成してください。この関数は出力を直接処理する必要があります。



上記の例は、対話型Pythonインタプリタ内で関数の結果が表示されたものです。この例をそのまま再現しようとしないでください。代わりに、`main.py`スクリプトを使用して関数をテストしてください。

第VI章

演習 1: 庭の名前

	演習1
ft_garden_name	
ディレクトリ: ex1/	
提出ファイル: ft_garden_name.py	
許可される関数: input(), print()	

「ft_garden_name」という名前の関数を記述し、庭の名前を入力してもらい、その名前と固定のステータスメッセージを表示するようにしてください。

```
>>> ft_garden_name()
庭の名前を入力してください: Community Garden庭:
Community Garden
ステータス: 順調に育っています！
```



メインプログラムではなく、関数 ft_garden_name() のみを記述してください。この関数は、入力と出力を直接処理する必要があります。



なお、「Status: Growing well!」は固定のメッセージであり、常に表示されている通り正確に表示される必要があります。

第 VII 章

演習 2: 庭の面積

	演習2
ft_plot_area	
ディレクトリ: ex2/	
提出ファイル: ft_plot_area.py	
使用許可: input(), int(), print()	

コミュニティガーデンでは、長方形の区画の面積を知る必要があります。長さと幅を入力を受け取り、面積を計算して表示する関数 `ft_plot_area` を記述してください。

```
>>> ft_plot_area() 長さを入力
してください: 5
幅を入力してください: 3
区画の面積: 15
```



メインプログラムではなく、関数 `ft_plot_area()` のみを記述してください。この関数は、入力と出力を直接処理する必要があります。

第VIII章

演習3：収穫総量

	演習3
ft_harvest_total	
ディレクトリ：ex3/	
提出ファイル：ft_harvest_total.py	
使用可：input()、int()、print()	

ある園芸家が、3日間にわたって野菜を収穫しました。各日の収穫量の重量を入力してもらい、合計を計算する関数 `ft_harvest_total` を記述してください。

```
>>> ft_harvest_total()1日目の収穫
量: 5
2日目の収穫量: 8
3日目の収穫量: 3
収穫総量: 16
```



メインプログラムではなく、関数 `ft_harvest_total()` のみを記述してください。この関数は、入力と出力を直接処理する必要があります。

第9章

演習4：植物の年齢確認

	演習4
ft_plant_age	
ディレクトリ：ex4/	
提出ファイル：ft_plant_age.py	
使用許可：input()、int()、print()	

ft_plant_age という名前の関数を記述し、植物の年齢（日数）を入力させ、収穫可能かどうか（60日より厳密に長い場合）を判定するようにしてください。

```
>>> ft_plant_age()
植物の年齢（日数）を入力してください: 75 植
物は収穫可能です！
>>> ft_plant_age()
植物の年齢（日数）を入力してください: 45 植
物はもう少し成長する時間が必要です。
```



メインプログラムではなく、関数 ft_plant_age() のみを記述してください。この関数は、入力と出力を直接処理する必要があります。

第X章

演習 5: 水やりリマインダー

	演習5
ft_water_reminder	
ディレクトリ: ex5/	
提出ファイル: ft_water_reminder.py	
使用可: input(), int(), print()	

ft_water_reminder という名前の関数を記述し、前回の水やりから何日が経過したかを尋ねるようにしてください。2日以上経過している場合は「Water the plants!」と出力し、そうでない場合は「Plants are fine」と出力してください。

```
>>> ft_water_reminder() 前回の水やり
から経過した日数: 4 植物に水をあげて
ください!
>>> ft_water_reminder() 前回の水やり
から経過した日数: 1 植物は大丈夫です
```



メインプログラムではなく、関数 ft_water_reminder() のみを記述してください。この関数は、入力と出力を直接処理する必要があります。

第XI章

演習6：収穫までの日数を数える

	演習6
ft_count_harvest	
ディレクトリ：ex6/	
提出ファイル：ft_count_harvest_iterative.py、ft_count_harvest_recursive.py	
許可される関数：input(), int(), print(), range(), 再帰のためのヘルパー関数の定義	

ft_count_harvest_iterative と ft_count_harvest_recursive という名前の関数を2つ作成してください。
どちらの関数も、1から指定された数値までを数え、収穫時期までの各日を表示するようにしてください。

```
>>> ft_count_harvest_iterative() 収穫までの日数:
5
1日目
2日目
3日目
4日目
5日目
収穫の時間だ！

>>> ft_count_harvest_recursive() 収穫までの日数
:5
1日目
2日目
3日目
4日目
5日目
収穫の時間だ！
```



メインプログラムではなく、関数 ft_count_harvest_iterative() と ft_count_harvest_recursive() のみを実装してください。これらの関数は、入出力を直接処理する必要があります。



再帰的なバージョンについては、以下のようなさまざまなアプローチが許容されます：

- メイン関数内でネストされたヘルパー関数を使用する
- デフォルトの引数値を使用する
- メイン関数から呼び出す独立したヘルパー関数を使用する

自分にとって最も理にかなっているアプローチを選んでください。再帰関数と反復関数の両方で、同じ出力が得られるはずです。

コードの拡張

ガーデンデータを通じたPythonの基礎

第XII章

演習7：型注釈を用いたシードの在庫管理

	演習7
ft_seed_inventory	
ディレクトリ：ex7/	
提出ファイル：ft_seed_inventory.py	
許可されている機能：print()、文字列メソッド	

菜園の責任者は、正確なデータ型を用いて種子の在庫を管理する必要があります。種子のパッケージを管理し、各種子の種類や数量に関する情報を表示する「ft_seed_inventory」という関数を記述してください。

```
>>> ft_seed_inventory("tomato", 15, "packets") トマトの種：15 パ  
ック在庫あり  
>>> ft_seed_inventory("carrot", 8, "grams") ニンジン  
の種：合計 8  
グラム  
>>> ft_seed_inventory("lettuce", 12, "area") レタ  
スの種：12平方メートル分をカバー
```

関数のシグネチャは、次のように記述する必要があります：

```
def ft_seed_inventory(seed_type: str, quantity: int, unit: str) -> None:
```



メインプログラムではなく、関数のみを記述してください。この関数は、入力と出力を直接処理する必要があります。



seed 型の先頭の大文字に注意してください。文字列オブジェクトで利用可能なメソッドを使用できます。

コードの拡張

ガーデンデータを通じたPythonの基礎



以下の単位をサポートしてください：「packets」（利用可能なパケット数）、「grams」（合計グラム数）、「area」（X平方メートルをカバー）。その他の単位については、その行に他の情報を含めず、「Unknown unit type」のみを出力してください。



このドキュメントの冒頭で述べたように、今後は Python の型ヒント（「typing」とも呼ばれる）を使用する必要があります。**mypy** を使用して型チェックを行ってください。

第XIII章 学習支援リソ

ース

学習をサポートするため、演習を簡単にテストできる main.py ファイルを用意しました。

Pythonを始めたばかりで、まだ main 関数の書き方がわからない方のために、このヘルパーは以下の機能を提供します：

- 演習用関数を自動的にインポートして実行します
- 問題が発生した場合は、わかりやすいエラーメッセージを表示します
- 個々の演習をテストしたり、まとめてテストしたりできます
- Pythonのインポートの仕組みを理解するのに役立ちます

このヘルパーを使用するには、ターミナルで ``python3 main.py`` を実行し、テストしたい演習を選択するだけです。演習用ファイル（例：ft_plot_area.py）が main.py と同じフォルダ内にあることを確認してください。



このヘルパーファイルは、読みやすく、学習に役立つように設計されています。動作原理を理解するためにコードを自由に確認しても構いませんが、まずは自分の演習の解答を書くことに集中してください。



ヘルパーファイルは学習支援のみを目的としています。提出する解答はご自身の作品であり、概念の理解を示しているものである必要があります。

第XIV章

提出と提出方法

課題は、通常通りGitリポジトリに提出してください。プレゼンテーションでは、リポジトリ内の作業内容のみが評価の対象となります。ファイル名が正しいかどうか、念のため再確認してください。



評価の際、コードの説明、実行の追跡、またはソリューションの修正を求められる場合があります。自分が書いたすべての行を理解していることを確認してください。



このプロジェクトでは、各課題で指定されたファイルのみを提出してください。Pythonの基礎を十分に理解していることを示す、整然として読みやすいコードを書くことに注力してください。